

Performance, Scalability, and Design Trade-Offs

Each enhancement above improves performance along one dimension, but introduces a corresponding cost. These trade-offs (not the mechanisms themselves) were the basis for each design decision.

Compound index: faster reads come at the cost of additional storage and slightly slower writes, since the index must be maintained on every insert, update, and delete. Because the dashboard performs far more reads than writes, this trade-off favors overall system performance.

Aggregation pipeline: shifting computation into MongoDB reduces network traffic and application-server load, but aggregation pipeline syntax is more complex to write and debug than equivalent client-side pandas code. The performance and bandwidth savings were judged to outweigh this added implementation complexity.

Pagination: `skip()/limit()` was chosen over cursor-based pagination for simplicity, at the cost of degrading performance on very large offsets. This was an acceptable trade given the dataset size in this application, but cursor-based pagination remains a better choice for datasets where deep pagination is common.

LRU cache: constant-time repeated-query performance comes at the cost of additional server memory and the need for correctness safeguards: specifically, cache invalidation on every `create`, `update`, and `delete`, to guarantee stale data is never served after a write.